

CLO1

Apply OOP concepts of inheritance, overloaded operators and polymorphism to simplify problem solving

Question#1:

- A. (15 marks) Given mystring class below, overload the operators to support the following main function.

```
class mystring {
    char arr[100];
public:
    mystring(char string[]){strcpy(arr, string);}
    void display(){cout<<arr;}
};

main(){
    mystring s1("abcd");
    mystring s2("xyz");
    mystring s3 = s1+s2;
    cout<<s3; //display s3 contents
    s1>>1; //if s1 is "abcd" then contents should change to "bcde"
}
```

Note that >> operator is not being overloaded for taking input. The right operand of >> is an int, which dictates how much offset each character will take.

```
mystring mystring::operator+(mystring &other){
    mystring temp;
    strcpy(temp.arr, arr);
    strcat(temp.arr, other.arr);
    return temp;
}

ostream& operator<<(ostream &out, mystring s){
    out<<s.arr;
    return out;
}

void mystring::operator>>(int i){
    for(int i=0; arr[i]!='\0'; i++)
        arr[i]++;
}
```

B. (5 marks) Overload the < operator to compare two dates for the following Date class.

```
class Date{
    int day;
    int month;
    int year;

    bool operator<(const Date& other) const {
        if (year < other.year)
            return true;
        if (year > other.year)
            return false;

        // Years are equal
        if (month < other.month)
            return true;
        if (month > other.month)
            return false;

        // Months are equal
        return day < other.day;
    }
}
```

CLO2

Demonstrate the use of basic linear data structures in problem solving

Question#2:

- A. (5 marks) Show what is produced by the following C++ code. Assume the node is the usual info-link form with the info of the type int.

Note that **list**, **trail**, and **current** are pointers of type nodeType

```
current = new nodeType;
current->info = 72;
current->link = NULL;
trail = current;
current = new nodeType;
current->info = 46;
current->link = trail;
list = current;
current = new nodeType;
current->info = 52;
list->link = current;
current->link = trail;
trail = current;
current = new nodeType;
current->info = 91;
current->link = trail->link;
trail->link = current;
current = list;
while (current!= NULL)
{
    cout << current->info << " ";
    current = current->link;
}
cout << endl;
```

46 52 91 72

- B. (10 marks) Remove error(s) in the following template function.

```
template <class type>
type funcExp(type list[], int size)
{
    type double x = list[0];
    type double y = list[size - 1];
    for (int j = 1; j < size ; j++)
    {
        if (x < list[j])
            x = list[j];
        if (y > list[size - 1 - j])
            y = list[size - 1 - j];
    }
    return y + x;
}
```

Suppose that you have the following declarations:

```
double sales[7] = {280.5, 320.0, 56.0, 78.90, 300.0, 100.0, 250.0};  
string names[] = {"Mike", "Lisa", "Nancy", "Robinson", "Miller", "Sam"};
```

What is the output of the following statements?

- `cout << funcExp(sales, 7) << endl;`

376

- `cout << funcExp(names, 6) << endl;`

LisaSam

C. (5 marks) Given a class Linklist, create a member function

```
void Linklist::displayFML()
```

which should print the values contained in the first node, middle node, and the last node of the link list.

```
void LinkList::displayFML(){  
    if(!head)  
        return;  
    cout<<"First:"<<head->info<<endl;  
    nodeType *ptr = head;  
    int size=0;  
    while(ptr->link){  
        size++;  
        ptr=ptr->link;  
    }  
    size++;  
    cout<<"Last:"<<ptr->info<<endl;  
    ptr=head;  
    for(int i=0;i<size/2;i++)  
        ptr=ptr->link;  
    cout<<"Middle:"<<ptr->info<<endl;  
}
```